Martin-Luther-Universität Halle-Wittenberg
Faculty of Natural Science II
Institute of Physics

Physics Modelling Assistant

Using Agentic Artificial Intelligence in physics modelling

Bachelor’s Thesis
for the Award of the Academic Degree
Bachelor of Science
in Physik und Digitale Technologien

submitted by: **Janis Felix Jacobi**
Supervisor 1: Prof. Dr. Niels Schröter
Supervisor 2: Prof. Dr. Samir Lounis

submission: Halle (Saale), 23rd August 2026

Contents

Figure Index

Table Index

Glossary

Agentic AI AI system with enhanced capabilities (subsubsection 2.1.3) 2, 3, 9

AI Artificial Intelligence 2–9

context-window a size describing how far back previous output token are considered in the LLM 3, 6

LLM Large Language Model 2, 3, 5–9

logit an LLMs output for every token indicating how well it is suited to be the next token. Larger values indicate a better fit. [5, p. 4] 3, 6, 7

token input for a LLMs transformation function 3, 5–9

1 Introduction

Artificial Intelligence can be of great assistance in scientific research [16]. As shown by Chugunova et al. [2], researchers in natural science are among the groups most familiar with AI among scientists. There is a wide range of application stretching from finding sources in the literature to aiding in the presenting research findings and including the steps in between. It can help in forming hypothesis and in evaluating them. This includes the design and control of practical experiments as well as aiding during simulations especially with the program implementation.[16]

To improve the work with AI in scientific research automated AI research pipelines where developed in the last two years. The first fully autonomous approach was published by Lu et al. in 2024[10]. Their framework called "The AI Scientist"[10] identifies interesting research questions, plans and executes according experiments, interprets the results and writes a paper stating its findings in a multistep process. Similar approaches where made by Team et al. with the "InternAgent"[14] which provides an interface for human feedback into the system. This is realised by the "Orchestration Agent"[14] which manages how and when human input is introduced into the system which itself consists of the four distinct tasks of "Survey", "Code Review", "Assessment" and "Idea Innovation"[14]. The "Orchestration Agent"[14] manages how the agents responsible for the interactions between the four tasks and how the findings are presented to the human. This system shows some resemblance to pure human research group with individuals specialized in specific parts of the work while a supervisor coordinates the process. The InternAgent was able to outperform the "AI-Scientist-V2" [17], a successor of "The AI Scientist"[10], as shown in the tests conducted by the InternAgent Team [14].

The publication of the InternAgent [14] also introduced measuring the costs of the AI usage during the process. The "InternAgent" [14] outperformed both instances of the "AI-Scientist" [10][17] at less than $\frac{1}{3}$ of the cost.

A less automated approach was made by Shao et al. with the introduction of "SciSciGPT" [13]. This system is for Science of Science research which is why SciSciGPT is equipped with data management component amongst a literature review and an analytics component. Similar to the "InternAgent" [14] [13] uses a component to handle the coordination of subtasks which is called "ResearchManager" [14] who gets the results of every subtask along with an evaluation from the "EvaluationSpecialist" [13], indicating if a rerun is needed. In contrast to the previously discussed frameworks "SciSciGPT" [13] does not generate its own research ideas, instead the human researcher makes a request (a text prompt) and the system takes this as its input. As mentioned by the authors Shao et al., "SciSciGPT" [13] is useful for more methodical tasks thereby allowing the human researcher to focus on the interpretation of the results.

As of 1st June 2026 the latest advancement in the field was made by Louapre, Niklaus, and Tunstall by presenting the "physics-intern"[9], "a multi-agent scaffolding system that takes a physics problem and works through it autonomously"[9]. They use a similar to previous systems, having an orchestration which manages a research component and a computation component. The results are reviewed and also checked against the literature before the results are feed back to the "Strategy Planner"[9]. The "Strategy Planner"[9] has access to a literature "Background Survey"[9] and can adjust the strategy which will be used by the orchestration.

The goal of this Thesis is to develop a system with a functionality similar to the "physics-intern"[9] (at the time the work on this Thesis began, the "physics-intern"[9] was not yet published). The system which is to be developed should take a physics problem/modelling request and craft a model and its implementation as an answer. Once this system is running, the influence of the systems configuration on its performance will be explored. With this research I want to contribute to a better understanding of AI and a more efficient use of it in scientific discovery.

2 Theoretical Foundations

Before going in to the details of AI based systems some definitions of important concepts will be made.

2.1 Definitions

2.1.1 Artificial Intelligence

Artificial Intelligence (AI) describes the functionality of a system. There is no universally accepted definition for AI. For the purpose of this Thesis, the definition from Russell and Norvig [12, p. 22-26] will be used: They describe AI as being a rational actor (rational agent), which acts in a way that achieves the best result based on the expectation of what the result should be ([12, p, 26], translated). This definition allows for a variety of systems to be considered an AI and accounts for the uncertainty regarding the goal the system should work towards.

2.1.2 Large Language Model

A Language Model is a "Transformer Language Model"[18] as described by Zhao et al. Then a multistep process is used to obtain an output:

1. The input text is divided into tokens, these are small words or parts of words. [18]
2. The input tokens are plugged into a transformation function which predicts the most likely next token based on its parameters, these include the previously generated tokens. The number of previous tokens considered is called context window. [7]

This results in a set of output tokens which are a human readable text and provide the most likely output which ideally is the result desired by the one providing the input. The process of developing a model which can do the described procedure well enough is a highly complex task and beyond the scope of this Thesis.

The distinction between Language Models and Large Language Models is in the number of internal parameters a model uses. Internal parameters include input and output tokens as well as the parameters discussed in subsubsection 2.2.3, subsubsection 2.2.4 and subsubsection 2.2.5. As stated in *A Survey of Large Language Models*[18], there is no hard cut-off for a Language Model being Large, but commonly models have around $10 \cdot 10^9$ parameters when considered Large Language Models (LLMs).

A LLM is one way of implementing AI.

2.1.3 Agentic AI

As described by Bandi et al., Agentic AIs are "autonomous, goal-driven systems that can operate on their own for long periods, requiring minimal human supervision"[1, section:3. Results]. In order to achieve this behaviour, these systems combine abilities that are beyond singular LLMs. Bandi et al. highlight the "strategic planning" [1, section:3. Results] capabilities, the preservation of the tasks context over time [1], the usage of "external tools to extend [the systems] capabilities" [1] and the collaboration of individual Agents (this means AI-systems) with other Agents[1].

2.2 Decoding the output of LLMs

In subsubsection 2.1.2 LLM is defined based on the functionality it provides. In order to understand how the behaviour of LLMs can be modified, a more formal definition will be given, based on the work of Ji et al. in *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers* [5].

2.2.1 LLM as a function which assigns probabilities

A LLM can be described as a function f with internal parameters θ. Additionally, f takes a set of input token $t_{in} = (i_1, ..., i_N)$ where $N \in \mathbb{N}$ is the number of input tokens, and a set of previous output token $v_{out,prev} = (o_{-1}, ..., o_{-M})$ where $M \in \mathbb{N}$ is the number of previous output tokens considered, which is called the context-window with size M. The function $f(\theta, t_{in}, t_{outp,prev})$ assigns a probability $s(v)$ to every token $v \in \mathcal{V}$ where $\mathcal{V}$ is the vocabulary of tokens known to the LLM:

$$f(\theta, t_{in}, t_{outp,prev}) = \begin{pmatrix} s(v'_1) \\ ... \\ s(v'_Z) \end{pmatrix} = s(v) \quad \text{where } Z \text{ is the size of the vocabulary } \mathcal{V} \tag{2.2.1}$$

v'_i with $i \in [1, Z] \subset \mathbb{N}$ is an element from the ordered set of $\mathcal{V}'$ which allows to map the probabilities to the . The probability indicates how well a token is suited to be the next output token[5, p. 4]. The probabilities $s(v)$ are called "logits"[5, p. 4] and not normalized by default, larger values of a logit signal a better fit for the corresponding token to be the next token.

The LLM outputs a probability distribution including every token from its vocabulary. To choose a token based on this probability distribution, Ji et al. state that the following optimization problem needs to be solved and labelled "MASTER PROBLEM"[5, p. 5]:

$$q^* = \arg \max_{q \in \Delta(\mathcal{V})} \left[\underbrace{\langle q, s \rangle}_{\sum_{v \in \mathcal{V}} q(v) \cdot s(v)} - \lambda \Omega(q) \right] \tag{2.2.2}$$

The used properties are:

- q^* is the probability distribution maximizing Equation 2.2.2
- $\Delta(\mathcal{V})$ is the set of all possible probability distributions
- q is a single probability distribution distributions for a given vocabulary $\mathcal{V}$
- s is the initial vector of logits the LLM produces
- $\Omega(q)$ is a regularisation function, which can push q^* towards a certain distribution
- λ is the strength of the regularisation function and $\lambda \geq 0$

There are different approaches for finding q^* and eventually getting a single token from it, the once relevant to this Thesis will be discussed in the following.

2.2.2 Greedy Decoding [5, p. 10]

This method uses no special regularisation function $\Omega(q)$, respectively sets $\lambda = 0$. This simplifies Equation 2.2.2 to $q^* = \arg\max_{q \in \Delta(\mathcal{V})} \langle q, s \rangle$. The resulting q^* vector has the value 1 in the line where s has its maximum, while every other line of q^* is 0. This means, that the next token will always be the most likely token according to the LLMs calculations. If multiple token have the highest probability $s(v)$, a single token is picked.[5, p. 10]

2.2.3 Softmax Decoding

For Softmax Decoding, the regularisation function is chosen as $\Omega(q) = -\sum_{v \in \mathcal{V}} q(v) \log(q(v))$ which is the negative entropy of q. Using this in solving Equation 2.2.2 leads to the following optimal distribution q^*, where $q^*(v)$ os the probability per token[5, p. 10-12]:

$$q^*(v) = \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}} e^{\frac{s(u)}{\lambda}}} \tag{2.2.3}$$

This distribution is normalised and has the parameter λ which controls how the distribution q^* is spread. For larger values of λ, the probability is spread more evenly across all tokens. This results in tokens the LLMs predicts to be less likely being more probable to be the next tokens. When λ becomes very small, the token the LLM calculates to me suitable become more likely. For the edge case $\lambda = 0$, simply Greedy Decoding is performed as it is discussed in subsubsection 2.2.2. The next token is then selected from the final probability distribution.

The parameter λ is one way of controlling the general randomness of the output. In the context of LLM it is often associated with the creativity of a model and called **Temperature** T [5, p. 10-12].

2.2.4 Top-k Decoding

Top-k Decoding extends Softmax Decoding (subsubsection 2.2.3) by selecting the k ($k \in \mathbb{N} \setminus k = 0$) token with the highest logit values[5, p. 12]. These token for a subset of the vocabulary called $\mathcal{V}_k$ on which Softmax Decoding is performed, while the probability for every other token is set to 0. This gives the following probability distribution[5, p. 12]:

$$q^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_k} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_k \\ 0 & \text{else} \end{cases} \tag{2.2.4}$$

For $k >= |\mathcal{V}|$, Top-k Decoding is equivalent to the standard Softmax Decoding (subsubsection 2.2.3). Setting $k = 1$ results in a result similar to Greedy Decoding (see subsubsection 2.2.2).

Top-k Decoding provides another parameter called **top_k** which can be used to shape the output of an AI system [5, p. 12].

2.2.5 Top-p Decoding

Top-p Decoding is another extension to Softmax Decoding (subsubsection 2.2.3) also selecting a subset of token called $\mathcal{V}_p$ from the vocabulary $\mathcal{V}$. In contrast the Top-k Decoding (subsubsection 2.2.4), Top-p Decoding defines a certain amount of probability $p \in (\,0, 1\,]$ which will be considered for the Softmax Decoding. How this works is that the token with the highest probability $s(v)$ from the set $\mathcal{V} \setminus \mathcal{V}_p$ will be added to the set $\mathcal{V}_p$. This will be repeated until the sum of the probabilities of all token $v \in \mathcal{V}_p$ exceeds the value of p. Then the probability of the tokens in $\mathcal{V} \setminus \mathcal{V}_p$ will be set to 0, while Softmax Decoding (subsubsection 2.2.3) will be performed on $\mathcal{V}_p$. The resulting probability distribution is (from [5]):

$$q^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_p} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_p \\ 0 & \text{else} \end{cases} \tag{2.2.5}$$

For $p = 1$ Top-p Decoding is equivalent to Softmax Decoding. For $p \to 0$, only the token with the highest probability gets a non 0 probability which makes this case equivalent to Greedy Decoding (subsubsection 2.2.2).

Top-p Decoding allows for dynamic adaptation based on how sharp or spread out the probability distribution given by the LLM is. For a sharp distribution, only the view very likely token are given a non 0 probability, whereas for a flatter distribution a higher number of tokens is selected and thus is a possible output token.

The resulting parameter **top_p** is yet another parameter often used in the field of LLM to modify a systems output [5, p. 13].

2.3 Other parameters that can influence an AI systems performance

2.3.1 Selection of the LLM

As shown by numerous benchmarks ([3], [4]) different LLM perform differently.

One of the most noticeable differences in LLM systems is the number of internal parameters used. The general assumption is that more parameters lead to better performance. Another important feature is the amount of data used to train a LLM. As shown by Kaplan et al.([6, p. 5]) combining the number of internal parameters N a model uses and the number of tokens D used in training a model can predict the "cross-entropy loss" [6, p. 6] L of a LLM:

$$L(N, D) = \left[\left(\frac{N_C}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_C}{D} \right]^{\alpha_D} \tag{2.3.1}$$

- $L(N, D)$ is the "cross-entropy loss"[6, p. 6]. Lower Values indicate a better model.
- N_C is the critical number of internal parameter until which Equation 2.3.1 is applicable. $N_C \approx 8.8 \cdot 10^{13}$
- N is the number of internal parameters the model uses
- D_C is the critical number of tokens until which Equation 2.3.1 is applicable. $D_C \approx 5.4 \cdot 10^{13}$
- α_N is the exponent for predicting $L(N)$. $\alpha_N \approx 0.076$
- α_D is the exponent for predicting $L(D)$. $\alpha_D \approx 0.095$

There are also other influences on a models performance, Kaplan et al. discus the influence of the number of training steps or the batch size (tokens per step) [6]. Liu et al. [8] showed the relevance of how the training data are composed, highlighting the increases in predicting a models performance.

2.3.2 Sources provided to a model

Since all LLMs have a limited amount of training data, the model on its own will not be able to answer some questions correctly. In order to mitigate this LLMs are getting fed extra information to help them answer the question at hand. Realizing this via human input is inefficient and impractical, especially when the human itself is not very knowledgeable in the question at hand. For that reason, LLM systems are getting equipped with tools, allowing them to access information not included in their training data. As shown by omeili, Shuster, and Weston [11] and Thoppilan et al. [15], equipping a LLM with some sort of web search tool allows to reduces the false output of the system. Note that tool use is one of the features making an AI an Agentic AI as discussed in subsubsection 2.1.3. sources provided; How many, which Execution time

2.4 Orchestrating AIs to behave as an Agentic AI

Apart from how the LLMs output is processed to arrive at a human readable output, other changes can be made to influence the AI systems behaviour. Some of these will be discussed in the following.

how agentic ai works

SAIA

Orchestration of Agentic ai

Crewai

Tools

3 Configurations of the pipeline

3.1 The chosen AI models

- PROBLEM: no clear benchmarks, only for older models or different versions of models => need to use general predictors: POWER SCALING LAW; Context window length; only lastly some general implication about the general capabilities of model families

===> this will be mostly a fixed choice since a real investigation of which model is suited better for which task would require an indepth analysis of how the models where designed and trained which is beyond the scope of this Thesis.

what to do instead: see whether assigning special models to certain tasks is better in the pipeline Design: Identify strongest overall model; identify supposetly strongest model in reasoning, argument handeling/filtering from large texts and coding - one run with only specialized models, one with overall best; compare; more as a prove of concept

Task & Agent defs

Steps

Output

4 Benchmarking

Figure 1: testC

Table 1: testTab

a	b
c	d

References

[1] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. "The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges". In: *Future Internet* 17.9 (2025). ISSN: 1999-5903. DOI: 10.3390/fi17090404. URL: https://www.mdpi.com/1999-5903/17/9/404 (cit. on pp. 5, 6).

[2] Marina Chugunova, Dietmar Harhoff, Katharina Hölzle, Verena Kaschub, Sonal Malagimani, Ulrike Morgalla, and Robert Rose. "Who uses AI in research, and for what? Large-scale survey evidence from Germany". In: *Research Policy* 55.2 (2026), p. 105381. ISSN: 0048-7333. DOI: https://doi.org/10.1016/j.respol.2025.105381. URL: https://www.sciencedirect.com/science/article/pii/S0048733325002100 (cit. on p. 4).

[3] HuggingFace. *Agent Arena*. URL: https://huggingface.co/spaces/lmarena-ai/arena-leaderboard (visited on 07/28/2026) (cit. on p. 8).

[4] HuggingFace, ed. *Artificial Analysis LLM Leaderboard*. URL: https://huggingface.co/spaces/ArtificialAnalysis/LLM-Performance-Leaderboard (visited on 07/28/2026) (cit. on p. 8).

[5] Xiaotong Ji, Rasul Tutunov, Matthieu Zimmer, and Haitham Bou-Ammar. *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers*. 2026. arXiv: 2602.18292 [cs.LG]. URL: https://arxiv.org/abs/2602.18292 (visited on 07/27/2026) (cit. on pp. 3, 6–8).

[6] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361 (visited on 07/28/2026) (cit. on pp. 8, 9).

[7] Vikram Krishnamurthy. *LLMs as High-Dimensional Nonlinear Autoregressive Models with Attention: Training, Alignment and Inference*. 2026. arXiv: 2602.00426 [cs.LG]. URL: https://arxiv.org/abs/2602.00426 (cit. on p. 5).

[8] Emmy Liu, Amanda Bertsch, Lintang Sutawika, et al. *Not-Just-Scaling Laws: Towards a Better Understanding of the Downstream Impact of Language Model Design Decisions*. 2026. arXiv: 2503.03862 [cs.CL]. URL: https://arxiv.org/abs/2503.03862 (visited on 07/28/2026) (cit. on p. 9).

[9] David Louapre, Joel Niklaus, and Lewis Tunstall. "physics-intern: an autonomous agentic framework for physics research". In: *Hugging Face* (04/12/2026). URL: https://huggingface.co/spaces/huggingface/physics-intern (visited on 07/24/2026) (cit. on p. 4).

[10] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: https://arxiv.org/abs/2408.06292 (cit. on p. 4).

[11] Mojtaba omeili, Kurt Shuster, and Jason Weston. *Internet-Augmented Dialogue Generation*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland, 05/2022. DOI: 10.18653/

v1/2022.acl-long.579. URL: https://aclanthology.org/2022.acl-long.579/ (visited on 07/28/2026).

[12] Stuart Russell and Peter Norvig. *Künstliche Intelligenz Ein moderner Ansatz*. Pearson Benelux B.V. (Zweigniederlassung Deutschland), 2023. ISBN: 9783868944303. URL: https://elibrary.pearson.de/book/99.150005/9783863263263 (visited on 07/26/2026) (cit. on p. 5).

[13] Erzhuo Shao, Yifang Wang, Yifan Qian, Zhenyu Pan, Han Liu, and Dashun Wang. "SciSciGPT: advancing human–AI collaboration in the science of science". In: *Nature Computational Science* (03/01/2026). URL: https://doi.org/10.1038/s43588-025-00906-6 (visited on 07/24/2026) (cit. on p. 4).

[14] InternAgent Team, Bo Zhang, Shiyang Feng, et al. *InternAgent: When Agent Becomes the Scientist – Building Closed-Loop System from Hypothesis to Verification*. 2025. arXiv: 2505.16938 [cs.AI]. URL: https://arxiv.org/abs/2505.16938 (cit. on p. 4).

[15] Romal Thoppilan, Daniel De Freitas, Jamie Hall, et al. *LaMDA: Language Models for Dialog Applications*. 2022. arXiv: 2201.08239 [cs.CL]. URL: https://arxiv.org/abs/2201.08239 (visited on 07/28/2026).

[16] Hanchen Wang, Tianfan Fu, Yuanqi Du, et al. "Scientific discovery in the age of artificial intelligence". In: *Nature* (08/01/2023). URL: https://doi.org/10.1038/s41586-023-06221-2 (visited on 07/24/2026) (cit. on p. 4).

[17] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search*. 2025. arXiv: 2504.08066 [cs.AI]. URL: https://arxiv.org/abs/2504.08066 (cit. on p. 4).

[18] Wayne Xin Zhao, Kun Zhou, Junyi Li, et al. *A Survey of Large Language Models*. 2026. arXiv: 2303.18223 [cs.CL]. URL: https://arxiv.org/abs/2303.18223 (cit. on p. 5).

5 Appendix

6 Declaration of authorship

I hereby declare that I have written this thesis independently and without unauthorized assistance.

All sources and references used have been properly cited.

This thesis has not been submitted in the same or substantially similar form for any other degree or academic qualification.

Halle (Saale), 23rd August 2026

Janis Felix Jacobi

7 Acknowledgement